# Federated Text-Based Chat Network & IRC Architecture

Compiled Technical Notes: IRC Bots, Multi-Server Linking, Decentralized Federation, and Multi-Language Linux Implementation

## 1. IRC Bots & Automation on Libera.Chat

You can run a bot on IRC and Libera.Chat to reply with preset messages and handle commands across multiple channels. On IRC, a bot is a client script or program connected to the server alongside regular users, provided it adheres to network bot policies.

### Popular Bot Software for Linux

- **Limnoria (Supybot).** A feature-rich Python-based IRC bot designed for multi-channel management. Its built-in `Factoids` and `Alias` plugins allow creating custom commands and automatic keyword replies without writing code.
- **Sopel.** A lightweight, modular Python IRC bot framework. It is easily installed on Linux via `pip` and lets developers write simple Python functions to trigger preset responses across specific or multiple channels.
- **Eggdrop.** One of the oldest and most stable C-based IRC bots. It uses Tcl scripts to handle channel moderation, custom `!commands`, and automated triggers.

### Libera.Chat Bot Requirements

- **Set Bot Mode (+B).** Set user mode `+B` on the bot before it joins any channels to flag the client as an automated bot in `WHO` and `WHOIS` queries.
- **Give the Bot Voice (+v) or Op (+o).** Channel operators should voice or op authorized bots so network anti-spam services like Ozone do not ban the bot for repetitive messages.
- **Use SASL Authentication.** Register a dedicated nickname with `NickServ` and authenticate via SASL.
- **Respect Rate Limits.** Configure message throttling so the bot does not flood channels when multiple users trigger commands simultaneously.
- **Support Channel.** Network guidance and testing help are available in the `#libera-bots` channel.

### Multi-Channel & Server Commands

- **Multi-Channel Replies.** Bots join a list of channels defined in their configuration (e.g., `["#channel1", "#channel2"]`) and can apply global triggers or restrict preset replies to specific channels.
- **Server-Level Commands.** A standard client bot cannot broadcast global messages across an entire public network. On public networks like Libera.Chat, bots reply only inside joined channels or via direct private messages ( `PRIVMSG` ).

## 2. Centralized Multi-Server IRC Networks (Server Linking)

Multiple IRC servers can be linked together into a single unified network and managed centrally. In IRC terminology, this is **server linking**, where all servers share the same global user list, channels, and nicknames.

### How IRC Server Linking Works

- **Tree Topology.** Traditional IRC networks connect servers in a loop-free spanning tree structure. Each server uses a unique Server ID ( `SID` ) and links to a central hub server.
- **Centralized Services.** Attaching an IRC Services suite (providing `NickServ` and `ChanServ` ) to the central hub syncs bans, registered nicknames, and operator permissions across all linked servers.
- **Global Operator Commands.** Network administrators (IRC Operators) can issue global commands such as `/KILL` , `/GLINE` (global ban), or `/WALLOPS` from any server to affect the entire network immediately.

### Popular IRCd & Services Software for Linux

- **InspIRCd + Anope.** InspIRCd is a modular C++ IRC daemon with XML `<link>` configuration blocks. Anope is a C++ IRC Services suite that provides central user registration, channel management, and global operator controls.
- **UnrealIRCd + Anope.** UnrealIRCd is a feature-rich Linux IRC daemon supporting cloaking and TLS server-to-server links, pairing natively with Anope or Atheme services.
- **Ergo (Standalone Option).** A modern IRC server written in Go with built-in services, designed primarily as a standalone single-server daemon rather than a linked multi-server network.

### IRC Server Linking vs. Open Federation

| Feature | Traditional IRC Network (Linked) | True Federation (e.g., Matrix, ActivityPub) |
|---|---|---|
| **Trust Model** | **High trust.** All linked servers must be explicitly configured and authorized by central administration. | **Open trust.** Any independent administrator can deploy a server and communicate with other servers. |
| **Control** | **Centralized.** One set of network operators and central services governs all servers. | **Decentralized.** Each server admin controls their own server, rules, and local user accounts. |
| **User Identity** | **Shared namespace.** Nicknames are global across the entire network (e.g., `Alice` ). | **Domain-scoped.** Users are scoped to their home server (e.g., `@alice:example.com` ). |

## 3. Designing a Custom Federated Text-Only Network

You can engineer a custom decentralized, text-only chat protocol combining ActivityPub-style federation with classic IRC line-based simplicity on Linux.

### Core Protocol Layers

- **Domain-Scoped Identities.** Federated users and rooms require server-qualified handles such as `alice@server-a.org` and `#general@server-a.org` to prevent namespace collisions across independent servers.
- **Client-to-Server (C2S) Protocol.** Keeping the client protocol line-delimited ( ) with classic verbs ( `NICK` , `USER` , `JOIN` , `PRIVMSG` , `PING` / `PONG` ) allows compatibility with terminal clients and lightweight CLI tools.
- **Server-to-Server (S2S) Federation.** When `alice@server-a.org` posts to `#general@server-b.org` , `server-a` discovers `server-b` via DNS SRV records or `.well-known` endpoints, establishes a TLS connection, and relays the signed frame.

### Distributed Systems Challenges

- **Authentication & Signatures.** Servers must verify message origins using public-key cryptography (such as Ed25519 signatures or HTTP Signatures) rather than trusting unverified plain-text sender claims.
- **Channel State & Moderation.** The host server of a channel (e.g., `server-a.org` for `#linux@server-a.org` ) acts as the authority for room topics, operator privileges, and bans, replicating state updates ( `MODE` , `KICK` , `TOPIC` ) to participating servers.
- **Network Splits (Netsplits) & Sync.** The protocol must define whether messages sent during a connection outage are dropped (ephemeral IRC model) or queued and replayed upon reconnection (persistent Matrix model).

## 4. Multi-Language Linux Implementation Stack

Using **GNU Assembler, C, C++, Go (Golang), GNU Guile Scheme Lisp, and Python** provides a clean division between low-level kernel networking, distributed S2S federation, and runtime scripting.

### Architectural Division of Responsibilities

- **C / C++ & GNU Assembler (Core Message Router & C2S Daemon).** Handles local TCP/TLS client sockets using Linux `epoll` or `io_uring` , zero-copy ring-buffer line parsing (with optional x86_64/AArch64 SIMD delimiter scanning in GNU Assembler), and in-memory channel fan-out.
- **Go / Golang (S2S Federation Gateway).** Runs as a dedicated federation worker daemon. Leverages Go's `net/http` , `crypto/tls` , and `crypto/ed25519` standard libraries for DNS SRV discovery, TLS handshakes, outbound delivery queues, and signature verification.

- **GNU Guile Scheme Lisp & Python (Scripting & Bot Layer).** Embeds `libguile` directly into the C/C++ core to expose C primitives (`send-privmsg`, `kick-user`, `register-hook`) for server-side logic and channel moderation. Python scripts can attach via CPython hooks or external Unix domain sockets for bots.

### Inter-Process Communication (C/C++ Core to Go Daemon)

- **Unix Domain Sockets (`AF_UNIX`).** Connects the C/C++ C2S engine and Go S2S worker over a local `SOCK_STREAM` or `SOCK_SEQPACKET` socket without Cgo overhead or internal TLS encryption costs.
- **Shared Memory Ring Buffer (`shm_open` + `mmap`).** Optional high-throughput path using a lock-free POSIX shared memory ring buffer paired with Linux `eventfd` notifications.

#### End-to-End Multi-Language Data Flow

1. **Local Client → C/C++ Core:** A client sends `PRIVMSG #dev@remote.org :hello` over a classic IRC-style socket.
2. **Core → Guile Scheme / Python:** The C/C++ daemon executes local channel hooks, moderation filters, or automated bot responses.
3. **Core → Go Federation Gateway:** Recognizing the remote domain `@remote.org`, the C/C++ core forwards the frame over an `AF_UNIX` socket to the Go worker.
4. **Go Gateway → Remote Server:** The Go daemon signs the payload with Ed25519, resolves `remote.org`, and delivers it over TLS to the remote federation endpoint.